

PROJECT REPORT

# AI Smart-Distiller & Workflow Automator

An Intelligent Information Pipeline

Extract · Structure · Automate

FAST-NUCES · ARTIFICIAL INTELLIGENCE · SPRING 2026

|               |                                   |
|---------------|-----------------------------------|
| Course        | Artificial Intelligence           |
| Instructor    | Atif Luqman                       |
| Institution   | FAST-NUCES Karachi                |
| Submission    | April 28, 2026                    |
| Group Members | Ghulam Mujtaba Qureshi — 24K-0535 |

|                   |                                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------------------|
|                   | Syed Muhammad Muzammil — 24K-0887                                                                               |
|                   | Muzammil Ali — 24K-1023 (Lead)                                                                                  |
| <b>Repository</b> | <a href="https://github.com/muzammil-zaidi/AI-Smart-Distiller">github.com/muzammil-zaidi/AI-Smart-Distiller</a> |

# 1. Executive Summary

AI Smart-Distiller is a multi-page Streamlit application that addresses the real-world problem of information overload faced by students and professionals. The system uses OpenAI's GPT-4o-mini model to extract structured insights — summaries, action items, and deadlines — from any long-form text, and automatically routes this data to Google Sheets and Gmail, eliminating the need for manual transcription.

The project was developed as a final submission for the Artificial Intelligence course at FAST-NUCES Karachi, demonstrating applied LLM usage, prompt engineering, direct API integration, and automated workflow design without relying on third-party automation platforms.

**~90%**

Time Saved

**4 Pages**

Multi-Page UI

**2 APIs**

Real Automation

## 2. Problem Statement

---

The core problem this project addresses is information overload — the phenomenon where students and professionals encounter large volumes of unstructured text daily and struggle to extract and act on key information efficiently.

### 2.1 The Manual Process

- A team receives 3–4 pages of meeting minutes after each session
- Someone must manually read and identify all action items
- Deadlines must be located and cross-referenced with calendars
- The summary must be written and distributed to stakeholders
- Data must be typed into tracking spreadsheets by hand
- The entire process takes 30–60 minutes per document and is error-prone

This inefficiency multiplies across an organization. A team handling 5 meetings per week loses approximately 150–300 person-hours per month to this manual process — time that could be directed toward productive work.

### 2.2 Proposed Solution

AI Smart-Distiller automates the entire pipeline in under 30 seconds:

- **Step 1:** User pastes any long-form text into the Distiller interface
  - **Step 2:** GPT-4o-mini analyzes the text and returns a structured JSON object
  - **Step 3:** The extracted data is displayed in organized tabs (Summary, Tasks, Deadlines)
  - **Step 4:** One click appends all data to a Google Sheet automatically
  - **Step 5:** One click sends a formatted HTML email report to any recipient
-

### 3. Project Objectives

---

- Build a functional multi-page Streamlit application with a professional UI
- Implement LLM-based structured data extraction using OpenAI's GPT-4o-mini
- Demonstrate practical prompt engineering to produce consistent JSON output
- Integrate directly with Google Sheets API using service account credentials
- Implement Gmail SMTP automation for formatted HTML email delivery
- Ensure the system degrades gracefully when API keys are not configured
- Provide a clear setup guide and documentation for all integrations

## 4. AI Approach & Methodology

---

### 4.1 Model Selection

The project uses OpenAI's GPT-4o-mini, selected for three reasons: (1) it is significantly cheaper than GPT-4o while maintaining strong NLU capabilities for structured extraction tasks; (2) it supports reliable JSON mode output; and (3) it has a 128K context window, sufficient for most long-form documents.

### 4.2 Prompt Engineering

The core AI innovation is the system prompt design. Rather than asking the model to 'summarize this text', the system prompt instructs GPT-4o-mini to perform a specific extraction task and return data in a strict, machine-parseable JSON format:

```
{  "summary": "2-3 sentence summary",  "tasks": ["Action item 1", ...],  "deadlines": [{  "date": "...",  "context": "..."  }]}
```

The prompt explicitly instructs the model to: return only JSON with no markdown fences, distinguish deadline dates from historical references, identify implicit action items not formatted as numbered lists, and keep the summary to exactly 2–3 sentences.

### 4.3 Why LLM Over Keyword Search

A regex-based approach would find every date in a document — including historical references ('founded in 1998') or examples ('for instance, on April 1st'). The LLM understands contextual intent: it knows that 'submit by May 1' represents a deadline while 'the policy was enacted in 2019' does not. This semantic understanding is the core AI contribution of this project.

### 4.4 Semantic Understanding & Attention

Internally, the transformer model employs multi-head self-attention across the full document context. Each token attends to all other tokens, allowing the model to resolve co-references (e.g., 'she will complete' → Sara will complete), understand organizational language (e.g., 'the deliverable is due') and correctly prioritize information by relevance rather than position.

---

## 5. System Architecture

### 5.1 Application Structure

The application is organized as a Streamlit multi-page app with the following file structure:

```
src/ Home.py           ← Landing page & system status  config.py           ← API
keys & settings  extractor.py       ← OpenAI API integration  sheets.py           ←
Google Sheets (gsread)  mailer.py       ← Gmail SMTP delivery  mock_mode.py
← Configuration helpers  pages/      1_Distiller.py       ← Text input & extraction UI
2_Automation.py       ← Push to Sheets & Email      3_About.py           ← Project info & setup
guide
```

### 5.2 Data Flow

The complete pipeline flows as follows:

- User pastes text into 1\_Distiller.py and clicks 'Run Distiller'
- extractor.py sends text + system prompt to OpenAI via API call
- GPT-4o-mini returns structured JSON: {summary, tasks, deadlines}
- Results are stored in st.session\_state and displayed in tabs
- User navigates to 2\_Automation.py and selects destinations
- sheets.py uses gsread + service account to append a row to Google Sheets
- mailer.py uses smtplib.SMTP\_SSL to send an HTML email via Gmail
- Status of each operation is displayed in a live status panel

### 5.3 Technology Stack

| Technology   | Version / Model | Purpose                      |
|--------------|-----------------|------------------------------|
| Python       | 3.12            | Core programming language    |
| Streamlit    | ≥ 1.33.0        | Multi-page web UI framework  |
| OpenAI API   | GPT-4o-mini     | NLU & JSON extraction        |
| gsread       | ≥ 6.0.0         | Google Sheets read/write     |
| google-auth  | ≥ 2.29.0        | OAuth2 service account auth  |
| smtplib      | Built-in        | Gmail SMTP email delivery    |
| Font Awesome | CDN             | UI icons for Streamlit pages |

## 6. Implementation Details

---

### 6.1 OpenAI Integration (extractor.py)

The extraction module initializes an OpenAI client using the key from config.py and makes a chat completion request. The temperature is set to 0.1 to minimize hallucination and maximize JSON consistency. The response is stripped of any markdown code fences and parsed with `json.loads()`. If parsing fails, the error is returned gracefully without crashing the application.

### 6.2 Google Sheets Integration (sheets.py)

The Google Sheets module uses `gsread` with a Google Service Account for server-to-server authentication. The module checks if the sheet is empty and writes a header row on first use. It then appends a single row containing the timestamp, summary, comma-joined tasks, formatted deadlines, and a 200-character source snippet. The service account email must have Editor access to the target spreadsheet.

### 6.3 Email Automation (mailer.py)

The email module uses Python's built-in `smtplib` with `SMTP_SSL` on port 465. Gmail requires an App Password (not the account password) for programmatic access. The email is composed as a `MIMEMultipart` alternative message with an HTML part, ensuring rich formatting in all email clients. The HTML template uses inline CSS for compatibility with Gmail, Outlook, and web mail clients.

### 6.4 Multi-Page Navigation

Streamlit's native multi-page architecture is used, with pages stored in the `pages/` directory. Session state (`st.session_state`) is used to pass the extracted JSON between the Distiller and Automation pages, avoiding re-extraction on navigation. Each page independently renders the sidebar navigation with consistent styling.

### 6.5 UI Design

The interface uses Google's Sora font for headings and JetBrains Mono for code elements, loaded via Google Fonts CDN. Material Icons Round (Google) are used for sidebar navigation icons. Font Awesome icons are injected via a helper function for in-page UI elements. The color scheme uses a dark navy palette (`#0c0c1d` background, `#e8d5b7` primary text) for a professional presentation-ready appearance.

---



## 7. Results & Evaluation

### 7.1 Functional Outcomes

- The Distiller correctly extracts summaries from meeting notes, articles, and project briefs
- Tasks are identified including implicit action items not in numbered lists
- Deadlines are distinguished from historical date references
- Google Sheets row is appended within 1–2 seconds of clicking Run Automation
- HTML email is delivered to inbox within 5–15 seconds via Gmail SMTP
- All 4 pages navigate correctly and share state via session\_state

### 7.2 Efficiency Comparison

| Step                      | Manual    | AI Smart-Distiller |
|---------------------------|-----------|--------------------|
| Read & summarize document | 10–15 min | < 5 sec (AI)       |
| Extract tasks & deadlines | 5–10 min  | < 5 sec (AI)       |
| Update Google Sheet       | 3–5 min   | ~1 sec (API)       |
| Email team                | 5 min     | ~10 sec (SMTP)     |
| <b>Total</b>              | 23–35 min | <b>&lt; 30 sec</b> |

### 7.3 Limitations

- GPT-4o-mini extraction requires a paid OpenAI API key with billing credits
- Google Sheets integration requires a Google Cloud project with APIs enabled
- Gmail SMTP requires 2-Step Verification and an App Password
- Very long documents (>50 pages) may exceed practical token limits
- The system does not handle scanned image-based PDFs (text input only)

## 8. Conclusion

---

AI Smart-Distiller successfully demonstrates the practical application of Large Language Models to a concrete real-world problem. By combining prompt engineering, direct API integration, and a polished multi-page Streamlit interface, the system delivers a measurable 90% reduction in the time required to process and distribute insights from long-form documents.

The project goes beyond displaying extracted text — it completes the full automation loop by pushing results to Google Sheets and Gmail SMTP. This design decision, replacing the originally planned n8n middleware with direct Python API integrations, demonstrates deeper technical understanding and produces a more reliable, dependency-free system.

The core AI contribution — using a transformer LLM's semantic understanding to distinguish contextual meaning — represents a genuine advancement over keyword-based approaches and illustrates the practical value of modern NLU in productivity applications.

---

## References

---

- [1] OpenAI. (2024). GPT-4o-mini Model Documentation. <https://platform.openai.com/docs>
- [2] Streamlit Inc. (2024). Streamlit Documentation. <https://docs.streamlit.io>
- [3] Anton Burnashev. (2024). gspread Documentation. <https://docs.gspread.org>
- [4] Google LLC. (2024). Google Sheets API Reference. <https://developers.google.com/sheets/api>
- [5] Python Software Foundation. (2024). smtplib — SMTP protocol client. <https://docs.python.org/3/library/smtplib.html>
- [6] Vaswani, A. et al. (2017). Attention Is All You Need. NeurIPS 2017.
- [7] Brown, T. et al. (2020). Language Models are Few-Shot Learners (GPT-3). NeurIPS 2020.